



12disk-io

Dönen Diskler (HDDs) ve Zamanın Matematiği

Hafıza Hiyerarşisi (Hız ve Maliyet Uçurumu)

Bilgisayardaki hafıza birimlerini bir piramit gibi düşün. İşlemciye (CPU) ne kadar yaklaşırsan hız inanılmaz artar ama kapasite küçülür ve pahalılaşır.

- **Zirve:** CPU Kayıtları (Registers) 1 nanosaniyede çalışır.
- **Orta Katman:** Ana Bellek (DRAM) 100 nanosaniyede cevap verir.
- **Taban (Depolama):** Bir Sabit Disk (HDD) 10 milisaniyede (yani 10.000.000 nanosaniyede) cevap verir.
Piramitten aşağı inilen her bir adım, bir öncekinden yaklaşık 10 kat daha yavaş ama 10 kat daha büyük/ucuzdur.

Plakçalar Anatomisi (HDD Fiziksel Yapısı)

Eski usul Sabit Diskleri (HDD), üzerine şarkılar kaydedilmiş üst üste duran eski plaklara ve bir plakçalara benzetebiliriz.

- **Plakalar (Platters):** Verilerin manyetik olarak yazıldığı, alüminyum veya camdan yapılmış yuvarlak disklerdir.
- **Mil (Spindle):** Bu plakaları saniyede 60 ila 250 tam tur hızla döndüren merkezdeki motordur.
- **Okuma/Yazma Kafası (Read/Write Head) ve Kol (Arm):** Plagın üzerinde ileri-geri gezinen ve veriyi okuyan iğnedir. Yüzeye asla fiziksel olarak değmez, milimetrik bir hava yastığı üzerinde uçar. (Eğer dokunursa "Head Crash" dediğimiz o meşhur fiziksel çizilme ve veri kaybı yaşanır).

İşletim sistemi bu yapıyı mantıksal olarak haritalandırır:

- En küçük veri paketine **Sektör** (Sector),
- Sektörlerin yan yana dizildiği halkalara **İz** (Track),
- Üst üste duran tüm plakaların aynı hizadaki izlerinin oluşturduğu hayali silindire ise **Silindir** (Cylinder) denir.

Zamanın Matematiği

Bir veriyi HDD'den okumak istediğinde araya giren fiziksel parçalar yüzünden 3 devasa zaman kaybı yaşanır:

1. **Arama Süresi (Seek Time):** Kolun, doğru izi (şarkıyı) bulmak için ileri veya geri hareket ettiği süredir. İçindeki en yavaş aşama budur.
2. **Dönüş Gecikmesi (Rotational Latency):** Kol doğru ize geldiğinde, diskin dönüp aradığımız sektörün tam iğnenin altına gelmesini beklediğimiz süredir.
3. **Aktarım Süresi (Transfer Rate):** Verinin kabloyla bilgisayara aktığı son süredir.

Aktarım süresi (Transfer Time), küçük veri blokları için diğer iki sürenin yanında "sıfır" kalacak kadar önemsizdir. Zamanın neredeyse tamamını diskin dönmesini ve kolun hareket etmesini bekleyerek kaybederiz.

Bu yüzden diskteki verileri yan yana okumak (**Sıralı / Sequential I/O**), kolu oradan oraya savuran dağınık okumadan (**Rastgele / Random I/O**) çok ama çok daha hızlıdır.

HDD'in Kontrol Edilmesi

Çözmemiz Gereken Sorun

Bilgisayarda arka planda virüs taraması çalışırken, sen de bir yandan oyun oynayıp arkada müzik dinliyorsan, işletim sistemine aynı anda onlarca farklı okuma/yazma isteği yağar. Disk kolunun (iğnenin) hareketi çok yavaş olduğu için, işletim sistemi bu istekleri gelişigüzel bir sırayla yapamaz. Amacımız iki şeyi dengelemektir: **Hız (Verim)** ve **Adalet (Kimseyi sonsuza kadar bekletmemek)**. Mühendisler bu dengeyi bulmak için tarih boyunca şu 4 algoritmayı geliştirmişlerdir:

Algoritmalar

1. FCFS (First Come First Served)

En saf ve ilkel yöntemdir. Siparişler hangi sırayla geldiyse, disk kolu o sırayla çalışır.

- Neden Kötüdür; Diyelim ki 1. istek diskin en başında, 2. istek en sonunda, 3. istek yine en başında. İğne bir silecek gibi diskin bir ucundan diğer ucuna sürekli savrulur durur. Adaleti mükemmeldir (kimse sırasını kaybetmez) ama performansı (hızı) felakettir.

2. SSTF / SPTF (En Kısa Arama Süresi İlk)

FCFS'nin yavaşlığından bıkan mühendisler bencil ve açgözlü (greedy) bir mantık kurarlar: *"Kol o an neredeyse, ona fiziksel olarak EN YAKIN olan isteği yapsın."*

- *Neden Patlar?* Hızı mükemmeldir, kol gereksiz yere sağa sola savrulmaz. Ancak ortada **Açlık (Starvation)** dediğimiz korkunç bir sorun doğar.

3. SCAN (Asansör Algoritması)

Açlık (Starvation) sorununu çözmek için binalardaki asansörlerin mantığı disk koluna kopyalanır. Kol diskin bir ucundan başlar, diğer ucuna doğru tek bir yönde ilerler ve yoldaki tüm verileri toplayarak gider. En sona (duvara) çarptığında yön değiştirir ve bu sefer geriye doğru süpürerek toplamaya başlar.

- *Neden Yetersizdir?* Orta kısımlar adaletsiz bir avantaj sağlar. Asansör sürekli aşağı yukarı inip çıkarken, orta katlardan iki kere geçer ama en alt/en üst katlardan sadece bir kere geçer. Yani diskin ortasındaki veriler, kenarındaki verilere göre çok daha hızlı okunur.

4. C-SCAN (Dairesel SCAN)

Orta kısımlara torpil geçilmesini engellemek için SCAN'in kuralı değiştirilir. Kol yine bir uçtan başlar ve yol boyunca verileri okuyarak en sona kadar gider. Fakat en sona geldiğinde, *geriye doğru okuyarak dönmez*; hiçbir şey okumadan hızla en başa atlar ve tekrar ileri doğru okumaya başlar (Tıpkı dairesel bir daktilo şeridi gibi). Bu sayede tüm bölgeler (baş, orta, son) tamamen eşit ve adil bir bekleme süresine kavuşur.

LOOK & C-LOOK Algoritmaları

SCAN ve C-SCAN algoritmalarına çok benzerdilerler, tek farkları diskin boş olan fiziksel son kısmına kadar gitmek yerine okuma yazma isteği olan son kısma kadar gidip oradan geri dönerler.

Modern İşletim Sistemlerinin Algoritmaları

Eskiden işletim sistemleri tek bir algoritma seçer ve onu kullanırdı. Ancak günümüzdeki modern sistemler (örneğin Linux) cihazın türüne göre farklı bir "trafik polisi" atar:

- **bfq (Bütçe Adil Kuyruklama):** Elinde dönen mekanik bir HDD varsa bunu kullanır, veri akışını adil dağıtmaya çalışır.
- **mq-deadline:** Elinde bir SSD varsa bunu kullanır. SSD'lerde hareket eden bir kol olmadığı için asansör mantığına gerek yoktur, sadece işlemlerin "son teslim tarihlerini" (deadline) takip ederek açlığı (starvation) önler.
- **kyber:** Süper hızlı NVMe cihazları içindir, kendi kendini ayarlayan çok basit bir yapısı vardır.

Katı Hal Sürücüler

SSD'lerin içinde dönen hiçbir parça yoktur, dolayısıyla "Arama Süresi" (Seek Time) veya "Dönüş Gecikmesi" (Rotational Latency) sıfırdır. HDD'ler saniyede 100-200 işlem (IOPS) yapabilirken, modern NVMe SSD'ler saniyede 1.000.000'dan fazla işlem yapabilir!

Ancak bu devasa hızın arkasında, işletim sistemini çıldırtabilecek kadar tuhaf fiziksel kurallar yatar.

1. Hücre Mantığı ve Üst Üste Binen Evler (NAND Flash)

SSD'ler veriyi, içine elektronların hapsedildiği "Hücrelerde" (NAND Flash Cell) saklar.

- **SLC (Single Level Cell - 1 Bit):** Bunu tek kişinin yaşadığı lüks bir stüdyo daire gibi düşün. İşlemler çok hızlıdır (kimse kimseyi beklemez) ve hücre çok dayanıklıdır (100.000 defa yazılıp silinebilir) ama maliyeti çok yüksektir.

- **MLC, TLC, QLC (2, 3 ve 4 Bit):** Maliyeti düşürüp kapasiteyi artırmak için aynı stüdyo daireyi paravanlarla bölüp içine 2, 3 veya 4 kişi sıkıştırırlar. Oda kalabalıklaştığı için veriyi okuyup yazmak yavaşlar ve hücrenin ömrü dramatik şekilde düşer (QLC'de ömür 1.000 silinmeye kadar düşer).

2. Üzerine Yazma Problemi

- Sadece bir **Sayfayı (Page)** (4 KB) mikrosaniyeler içinde çok hızlı okuyabilirsin.
- Boş bir **Sayfaya** çok hızlı veri yazabilirsin.
- **AMA dolu bir sayfanın üzerine doğrudan yeni veri YAZAMAZSIN!** Eski veriyi silmen gerekir.
- İşin korkunç yanı: Bir sayfayı tek başına silemezsin. Silme işlemi (Erase) ancak yüzlerce sayfanın birleşiminden oluşan devasa bir **Blok (Block)** düzeyinde yapılabilir!. Yani tek bir kelimeyi düzeltmek için bütün bir kitabı ateşe atman gerekir.

3. SSD'nin Gizli Kahramanı: FTL (Flash Translation Layer)

Eğer işletim sistemi (Windows/Linux) her veri güncellediğinde bu silme kuralıyla uğraşsaydı, bilgisayarın anında donardı. Bu yüzden SSD'lerin içine işletim sistemini "kandıran" harika bir donanımsal beyin eklenmiştir: **FTL (Flaş Çeviri Katmanı)**.

Sen küçük bir dosyayı (örneğin bir Word belgesini) güncellediğinde FTL arka planda şunları yapar:

1. Eski verinin üzerine yazmaz. Güncel dosyayı SSD'de tamamen *boş* olan yepyeni bir sayfaya yazar.
2. İşletim sisteminin adres defterini (Mapping Table) güncelleyerek, "Dosyanın yeni yeri burası" der.
3. Eski verinin bulunduğu sayfaya "Geçersiz (Invalid) / Çöp" işaretini koyar ve bırakır.

4. Temizlik İşçisi ve Yıpranma (GC ve Wear Leveling)

Zamanla SSD'nin içi bu "Çöp" sayfalarla dolar ve boş yer kalmaz.

- **Çöp Toplama (Garbage Collection):** FTL, içinde çok fazla çöp birikmiş o koca "Blokları" tespit eder. İçindeki hâlâ geçerli olan (sağlam) 3-5 sayfayı alır,

yepyeni bir bloğa kopyalar. Sonra o eski bloğun tamamını toptan SİLER (Erase). Böylece yepyeni tertemiz sayfalar elde ederiz.

- **Yıpranma Dengelemesi (Wear Leveling):** Hücrelerin sürekli silinip yazılmaktan ömrü biter (ölür). FTL, diskin hep aynı yeri ölmesin diye, yeni gelen verileri sürekli diskin farklı fiziksel köşelerine dağıtarak yıpranmayı tüm diske eşit yayar.

5. Performansın Düşmanı: Yazma Yükseltmesi (Write Amplification)

Disk in ağzına kadar doluysa ve sen sadece 4 KB'lık tek bir sayfa kaydetmek istersen sistem tıkanır. FTL sana yer açmak için çöp toplamaya başlar; belki de o 4 KB için arkada koca bir bloğu (512 KB) okuyup başka yere taşımak ve silmek zorunda kalır. Sen sadece 1 sayfa yazdığını sanırsın ama SSD fiziksel olarak 100 sayfalık amelelik yapar. Buna **Yazma Yükseltmesi** diyoruz ve SSD'yi hem yavaşlatıp hem de ömrünü bitiren ana sorundur.

Çoklu Disklerin Yönetilmesi

1. Cihazları bağlama

Diyelim ki bir şirketin binlerce terabayt verisi var. Bu diskleri bilgisayarlara nasıl bağlayacağız? Üç ana mimari vardır:

1. **Host-Attached (Doğrudan Bağlı):** Klasik yöntemdir. Diski alıp kasanın içine (PCIe vb.) takarsın.
2. **NAS (Network-Attached Storage - Ağa Bağlı Depolama):** Evindeki modeme bağladığın bir yedekleme disk gibi düşün. İşletim sistemine sanki **ağdaki paylaşılan bir "Klasör" (Dosya Sistemi)** gibi görünür. İnternet kabloları üzerinden (NFS, CIFS protokolleriyle) haberleşir.
3. **SAN (Storage Area Network - Depolama Alanı Ağı):** Devasa veri merkezlerinde kullanılır. NAS'taki gibi normal bir ağ üzerinden bağlarsan çok yavaşlar. Bu yüzden SADECE disklere özel, ultra hızlı fiber optik ayrı bir ağ kurulur. İşin en can alıcı kısmı şudur: SAN üzerindeki bir disk işletim sistemine

ağ klasörü gibi değil, sanki **kasanın içine doğrudan takılmış ham bir Sabit Disk (Block)** gibi görünür!

2. RAID

Disk in bozulmasına (MTTF - Arızaya Kadar Geçen Ortalama Süre) karşı bir önlem almamız gerekir. Diskleri tek tek kullanmak yerine, işletim sistemini kandırıp 10 tane fiziksel diski tek bir mantıksal diskmiş gibi kullanma sanatına **RAID** diyoruz. İki amacı vardır: **Güvenilirlik (Yedekleme)** ve **Performans (Paralel Hız)**.

RAID Seviyeleri

- **RAID 0 (Şeritler - Striping):** Hızın zirvesidir. Elinde 100 MB'lık bir dosya var. RAID 0 bunu ikiye böler, 50 MB'ını 1. diske, 50 MB'ını 2. diske *aynı anda* yazar. İşlem yarı sürede biter! Ancak çok büyük bir bedeli vardır: Disklerden biri bile bozulursa, dosyanın sadece yarısı kalacağı için verinin tamamı çöp olur (Yedekleme/Koruma SIFIRDIR).
- **RAID 1 (Aynalama - Mirroring):** Güvenliğin zirvesidir. Aynı dosyayı iki diske de kelimesi kelimesine yazar (Kopya çıkarır). Disklerden biri cayır cayır yansa bile diğeri sapasağlamdır. Dezavantajı: Çok pahalıdır. 2 TB'lık iki disk alırsın ama elinde kullanabileceğin sadece 2 TB alan olur, çünkü yarısı kopyalamaya (israfa) gider.
- **RAID 5 (Dağıtılmış Parite - Parity):** Zekânın ve mühendisliğin zirvesidir. En az 3 diskle yapılır. Dosyayı disklere bölerken aralara "Parite" (P) adı verilen matematiksel bir denklem sonucu sıkıştırır. Disklerden biri aniden bozulduğunda sistem paniklemez. Diğer sağlam disklerdeki kalan verileri ve o matematiksel denklem sonucunu (P) kullanarak, yanan diskteki veriyi **havadan anında geri hesaplar!**. Hem yer tasarrufu sağlar hem de güvenlidir.
- **RAID 6 (Çift Parite):** RAID 5 gibidir ama aralara ekstra bir denklem (Q) daha koyar. Böylece aynı anda 2 disk birden yansa bile sistemi kurtarabilir.
- **RAID 10 ve 01 (Melez Sistemler):** RAID 0'ın (Hız) ve RAID 1'in (Aynalama) birleşimidir. Şeritlere böldüğü veriyi aynı anda aynalar. Performansı ve güvenliği muazzamdır ama çok fazla diske mal olur.

3. Yeni Nesil Veri Korumaları

ZFS

Klasik RAID sistemleri bir diskin komple yandığını anlar ama diskin içindeki 1 baytlık verinin zamanla manyetik olarak "sessizce bozulduğunu" (çürüdüğünü) anlayamaz. Solaris ZFS dosya sistemi, her bir veriye ve adrese **"Sağlama Toplamı" (Checksum)** ekler. Veri okunurken bozulduğu anında fark edilir ve diğer sağlam kopyadan orijinal veri anında düzeltilir.

Nesne Depolama

Günümüzde (özellikle Hadoop, Ceph gibi sistemlerde) devasa verileri saklamak için klasör, dosya ve ağaç mantığı yetersiz kalır. Veriler, dipsiz, hiyerarşisi olmayan bir havuza "Nesne" (Object) olarak atılır ve hepsine benzersiz bir kimlik numarası (ID) verilir. İsteyen program, ID'yi verip verisini çeker.